COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION: CPE21S2**	
DATE PERFORMED:	7/24/26	**DATE SUBMITTED:**	7/29/26
NAME: ESTORCO, CLINT HOVEN A.		**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 1.1 — **Seatwork 4.1 Array based vs STL Stack implementation**

1. PROCEDURE OUTPUT

SCREENSHOT	

```cpp
#include <iostream>
#include "doubly_ll.h"

int main() {

    DoubleList<char>* head = CreateNewNode('C');
    DoubleList<char>* second = CreateNewNode('P');
    DoubleList<char>* third = CreateNewNode('E');

    head->next = second;
    second->prev = head;

    second->next = third;
    third ->prev = second;

    std::cout<<"Testing the DLL Traversal"<<std::endl;
    dllTraverse(head);

std::cout<<"Testing the insertion at the head node"<<std::endl;
    dllInsertHead('X', &head);
    dllTraverse(head);

    std::cout<<"Testing the insertion at the end node"<<std::endl;
    dllInsertEnd('0', head);
    dllTraverse(head);

std::cout<<"Testing the insertion at the General Insert node"<<std::endl
    dllGenInsert('6', head -> next-> next);
    dllTraverse(head);

std::cout<<"Testing the insertion of the Delete node"<<std::endl;
    dllDelNode('P', &head);
    dllTraverse(head);

    return 0;
}
```

DISCUSSION	This program will test a custom Doubly Linked List with dynamically generated nodes that have character data ('C', 'P', 'E'). It then systematically shows how to traverse a list and how to perform key operations like adding nodes to the head, middle and end of a list and removing a node from the list at the target.

OPERATION	SCREENSHOT

TRAVERSAL	`DoubleList<char>* head = CreateNewNode('C');`
	`DoubleList<char>* second = CreateNewNode('P');`
	`DoubleList<char>* third = CreateNewNode('E');`
	
	`head->next = second;`
	`second->prev = head;`
	
	`second->next = third;`
	`third->prev = second;`
	
	`std::cout<<"Testing the DLL Traversal"<<std::endl;`
	`dllTraverse(head);`

INSERTION AT THE HEAD	`std::cout<<"Testing the insertion at the head node"<<std::endl;`
	`dllInsertHead('X', &head);`
	`dllTraverse(head);`

INSERTION AT ANY PART OF THE LIST	`std::cout<<"Testing the insertion at the end node"<<std::endl;`
	`dllInsertEnd('O', head);`
	`dllTraverse(head);`

INSERTION AT THE END	`std::cout<<"Testing the insertion at the end node"<<std::endl;`
	`dllInsertEnd('O', head);`
	`dllTraverse(head);`

DELETION OF NODE	`std::cout<<"Testing the insertion of the Delete node"<<std::endl;`
	`dllDelNode('P', &head);`
	`dllTraverse(head);`

A	Source Code	

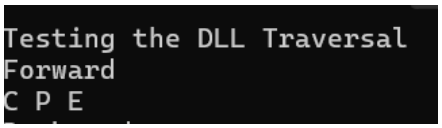
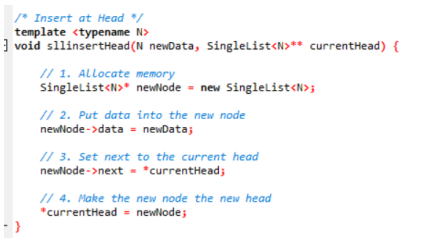
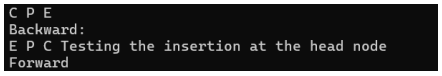
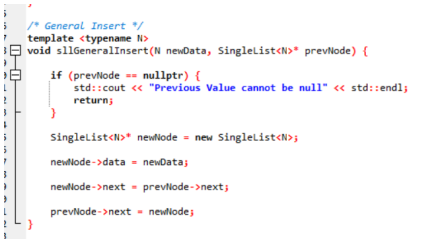
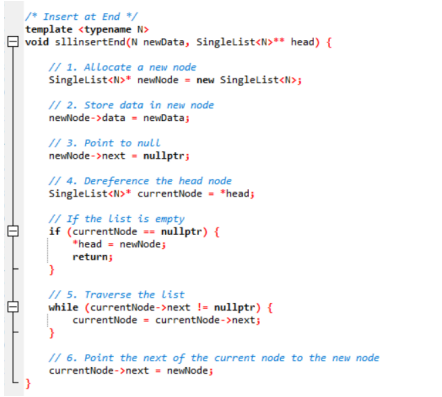
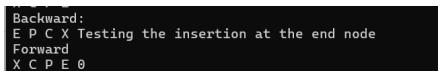
```
/* List Traversal */
template <typename N>
void ListTraversal(SingleList<N>* head) {

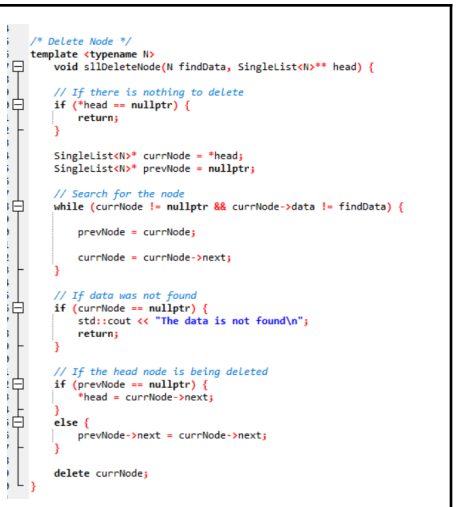
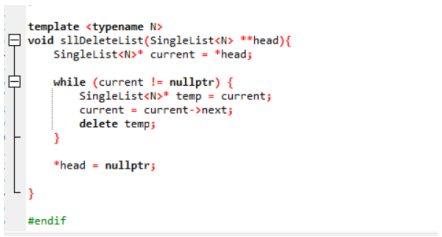
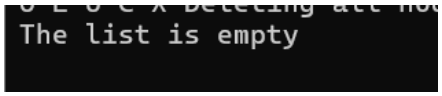
    while (head != nullptr) {

        std::cout << head->data << "-->";


        head = head->next;
    }

    std::cout << std::endl;
}
```

	Console	
B	**Source Code**	
	Console	
C	**Source Code**	
	Console	
D	**Source Code**	
	Console	

E	**Source Code**	
	Console	
F	**Source Code**	
	Console	

```
/* Delete Node */
template <typename N>
    void sllDeleteNode(N findData, SingleList<N>** head) {

    // If there is nothing to delete
    if (*head == nullptr) {
        return;
    }

    SingleList<N>* currNode = *head;
    SingleList<N>* prevNode = nullptr;

    // Search for the node
    while (currNode != nullptr && currNode->data != findData) {

        prevNode = currNode;

        currNode = currNode->next;
    }

    // If data was not found
    if (currNode == nullptr) {
        std::cout << "The data is not found\n";
        return;
    }

    // If the head node is being deleted
    if (prevNode == nullptr) {
        *head = currNode->next;
    }
    else {
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

```
Backward:
0 E 6 Testing the insertion of the Delete node
Forward
```

```
template <typename N>
void sllDeleteList(SingleList<N> **head){
    SingleList<N>* current = *head;

    while (current != nullptr) {
        SingleList<N>* temp = current;
        current = current->next;
        delete temp;
    }

    *head = nullptr;

}

#endif
```

```
The list is empty
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

INPUT	OUTPUT

```cpp
#include <iostream>
#include <string>

// Node structure to represent each song
struct SongNode {
    std::string title;
    SongNode* next;
    SongNode* prev;

    SongNode(std::string name) {
        title = name;
        next = NULL;
        prev = NULL;
    }
};
// Circular Doubly Linked List class for Playlist
class Playlist {
private:
    SongNode* head;
    SongNode* current;

public:
    Playlist() {
        head = NULL;
        current = NULL;
    }
    // Add a song to the end of the circular playlist
    void addSong(std::string title) {
        SongNode* newSong = new SongNode(title);

        if (head == NULL) {
            head = newSong;
            head->next = head;
            head->prev = head;
            current = head;
            std::cout << "\"" << title << "\" added as the first song.\n";
            return;
        }
        SongNode* tail = head->prev;

        tail->next = newSong;
        newSong->prev = tail;
        newSong->next = head;
        head->prev = newSong;

        std::cout << "\"" << title << "\" added to the playlist.\n";
    }
    // Remove a song by title
    void removeSong(std::string title) {
        if (head == NULL) {
            std::cout << "Playlist is empty! Cannot remove.\n";
            return;
        }
        SongNode* temp = head;
        bool found = false;
        // Search for the song
        do {
            if (temp->title == title) {
                found = true;
                break;
            }
            temp = temp->next;
        } while (temp != head);

        if (!found) {
            std::cout << "Song \"" << title << "\" not found in playlist.\n";
            return;
        }

        // Case 1: Only one node in list
        if (temp->next == temp) {
            delete temp;
            head = NULL;
            current = NULL;
            std::cout << "Removed \"" << title << "\". Playlist is now empty.\n";
            return;
        }

        // Case 2: Removing the head node
        if (temp == head) {
            head = head->next;
        }

        // Case 3: Update pointer references if current song is deleted
        if (temp == current) {
            current = current->next;
        }

        temp->prev->next = temp->next;
```

```
"You give love a bad name" added as the first song.
"Bed of Roses" added to the playlist.
"In these Arms Tonight" added to the playlist.

==============================
   SPOTICLINT
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 1
Enter song title: Living on a Prayer
"Living on a Prayer" added to the playlist.

==============================
   SPOTICLINT
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 2
Enter song title to remove: In these Arms Tonight
Successfully removed "In these Arms Tonight".
```

```
================================
   SPOTICLINT
================================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 3
Now Playing: Bed of Roses

================================
   SPOTICLINT
================================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 4
Now Playing: You give love a bad name
```

```
        temp->next->prev = temp->prev;

        delete temp;
        std::cout << "Successfully removed \"" << title << "\".\n";
    }
    // Play next song
    void playNext() {
        if (current == NULL) {
            std::cout << "Playlist is empty.\n";
            return;
        }
        current = current->next;
        std::cout << "Now Playing: " << current->title << "\n";
    }
    // Play previous song
    void playPrev() {
        if (current == NULL) {
            std::cout << "Playlist is empty.\n";
            return;
        }
        current = current->prev;
        std::cout << "Now Playing: " << current->title << "\n";
    }
    // Print all songs once starting from head
    void playAll() {
        if (head == NULL) {
            std::cout << "Playlist is empty.\n";
            return;
        }
        std::cout << "\n--- Playlist Songs ---\n";
        SongNode* temp = head;
        int trackNum = 1;

        do {
            std::cout << trackNum << ". " << temp->title;
            if (temp == current) {
                std::cout << "  <-- [Currently Selected]";
            }
            std::cout << "\n";
            temp = temp->next;
            trackNum++;
        } while (temp != head);

        std::cout << "----------------------\n";
    }
};

int main() {
    Playlist myPlaylist;
    int choice;
    std::string songTitle;
    // Pre-populate with a few songs
    myPlaylist.addSong("You give love a bad name");
    myPlaylist.addSong("Bed of Roses");
    myPlaylist.addSong("In these Arms Tonight");

    while (true) {
        std::cout << "\n==============================\n";
        std::cout << "   SPOTICLINT \n";
        std::cout << "==============================\n";
        std::cout << "1. Add Song\n";
        std::cout << "2. Remove Song\n";
        std::cout << "3. Next Song\n";
        std::cout << "4. Previous Song\n";
        std::cout << "5. Display All Songs\n";
        std::cout << "6. Exit\n";
        std::cout << "Enter choice: ";
        std::cin >> choice;
        std::cin.ignore(); // Clear input buffer

        switch (choice) {
            case 1:
                std::cout << "Enter song title: ";
                std::getline(std::cin, songTitle);
                myPlaylist.addSong(songTitle);
                break;
            case 2:
                std::cout << "Enter song title to remove: ";
                std::getline(std::cin, songTitle);
                myPlaylist.removeSong(songTitle);
                break;
            case 3:
                myPlaylist.playNext();
                break;
            case 4:
                myPlaylist.playPrev();
                break;
            case 5:
```

```
==============================
   SPOTICLINT
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 5

--- Playlist Songs ---
1. You give love a bad name  <-- [Currently Selected]
2. Bed of Roses
3. Living on a Prayer
----------------------

==============================
   SPOTICLINT
==============================
1. Add Song
2. Remove Song
3. Next Song
4. Previous Song
5. Display All Songs
6. Exit
Enter choice: 6
Exiting music player...

--------------------------------
```

```
178                 myPlaylist.playAll();
179                 break;
180             case 6:
181                 std::cout << "Exiting music player...\n";
182                 return 0;
183             default:
184                 std::cout << "Invalid choice! Try again.\n";
185         }
186     }
187     return 0;
188 }
```

CONCLUSION:

To conclude it, the program makes use of a circular doubly linked list that could be used in the music player playlist implementation to achieve a continuous traversal without encountering a dead-end pointer. With a dedicated current pointer, it is possible to use just a prev and a next reference, thus providing efficient state management for selecting or swapping songs. As a whole, this activity illustrates how the circular structures safely insert and delete nodes while maintaining continuous memory linkages as appropriate for loop-style applications.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY